Midterm Practice Questions - Solutions

Add 1

A random variable X is a member of an exponential family if its density $f(x|\theta)$ can be written as:

$$f(x|\theta) = a(\theta)b(x)\exp\left[c(\theta)d(x)\right],$$

where $a()$ and $c()$ are functions only of the parameter θ, and $b()$ and $d()$ are functions of x.

Verify that the Binomial distribution is an exponential family. Note, $X \sim Bin(n, p)$ has pmf given by:

$$P(X = x|p) = (n \text{ choose } x)p^x(1-p)^{n-x}.$$

SOLUTION

This is very similar to the Bernoulli example on the chapter 8 handout.

We start from the pmf and do some rearranging.

$$P(X = x|p) = (n \text{ choose } x)p^x(1-p)^{n-x}.$$

$$P(X = x|p) = (1-p)^n \ (n \text{ choose } x)\left(\frac{p}{1-p}\right)^x.$$

We can see the a() and b() parts already, but need to get the last part into an exp to finish.

$$P(X = x|p) = (1-p)^n \ (n \text{ choose } x)\exp\left[x\log\left(\frac{p}{1-p}\right)\right].$$

So, $a(p) = (1-p)^n$, $b(x) = (n \text{ choose } x)$, $c(p) = log(p/(1-p))$ and $d(x) = x$. Thus, the Binomial distribution is an exponential family.

Add 2

Describe how to obtain a bootstrap distribution for a chosen statistic and obtain a (say) 95 percent confidence interval for its related parameter of interest.

SOLUTION

We start with a data set of n observations with p variables. A bootstrap sample is a sample of n observations selected WITH replacement from the set of n observations in our data set. All p variables are retained. Some of the original observations will be repeated and others will be left out. We compute our statistic on the original data set, and then compute the statistic again on B bootstrapped samples. B is usually 1000 or more. This gives us a bootstrap distribution for the chosen statistic - a distribution of values for the statistic from "similar" samples.

Once obtained, what are some of the uses of a bootstrap distribution? Well, one use is to get a confidence interval for the related parameter (whatever the statistic is estimating). Assuming the bootstrap distribution of the statistic is not too skewed, we can just take appropriate quantiles. I.E. for a 95 percent CI, take the 2.5 and 97.5 quantiles of the bootstrap distribution.

Alternatives based on normal-shaped or skewed distributions were explored in Stat 230. But the quantile idea is a basic approach that is widely applicable. If you are interested in learning more about the bootstrap, Chapter 11 in CASI has more info and could lead to topics you explore for the final project.

Add 3

Explain the plug-in principle in relation to obtaining the standard error of a sample proportion.

SOLUTION

Working from properties of the binomial distribution for a RV X $Bin(n, p)$, we can find that $\hat{p} = X/n$ has a standard deviation of $\sqrt{p(1-p)/n}$. We do not usually know the value of p though, hence why we are interested in $\hat{p}$ to begin with. The plug-in principle says that we can use $\hat{p}$ in place of p in the formula and obtain the standard error of $\hat{p}$, which is our best estimate of the standard deviation of $\hat{p}$.

Hence, we find that $SE(\hat{p}) = \sqrt{\hat{p}(1-\hat{p})/n}$.

Add 4

Explain what a Bayesian conjugate family is and what its benefits are.

SOLUTION

Bayesian conjugate families describe situations where the distributions for the prior and data are "nice" interacting with each other such that the posterior is from the same family as the prior, but with altered parameters (basically, updated based on the data).

Examples: You had a Beta-Binomial (prior is Beta, data observed was Binomial (or independent Bernoullis)) on your homework. There is also a Normal-Normal (Normal prior for the mean of the normal model for the data), a Gamma-Poisson (Gamma for the Poisson parameter λ) and others covered in Stat 370.

The major benefit of working in these families is the easy (tractable) math.

Add 5

Using the Chapter 8 Lab data set for context, explain how to perform a permutation test to see whether the mean age differs for individuals with monthly income > 7500 versus individuals with less than that monthly income.

```
credit <- read.csv("https://awagaman.people.amherst.edu/stat495/creditsample.csv",
                   header = T)
```

SOLUTION

We have a group variable that indicates whether individuals have a monthly income > 7500 versus not. (Equal to 7500 should go in one group, so putting in less than group.)

```
credit <- mutate(credit, group = ifelse(MonthlyIncome > 7500, 1, 0))
tally(~ group, data = credit)
```

```
group
    0     1  <NA>
16862  7183  5955
```

We have 16862 individuals with income <= 7500 and 7183 with income > 7500. We ignore NAs here (you can argue that you want to include them in the shuffling as well).

We take the 24045 individuals with income recorded and record their average age by group.

```
favstats(age ~ group, data = credit)
```

```
  group min Q1 median Q3 max    mean      sd     n missing
1     0  21 39     50 62 102 50.5875 15.2925 16862       0
2     1  24 45     53 61  97 53.1694 11.4434  7183       0
```

We see that the higher income group has a slightly higher mean age. But is this statistically significant? The permutation test helps us address this question.

With an appropriate seed set for reproducibility, we take the same individuals and shuffle their group labels, recomputing the average ages per group. We can save some summary measure of that difference - perhaps the t-test statistic that would result or the difference in means itself. We repeat this process. This generates a distribution of summary measures that would be possible under the null hypothesis of no relationship of age with income group. Once we have an appropriate distribution built up, we assess how unusual our observed measure on the original sample was in comparison to the distribution. If we have a value in the tails, it suggests that our results would not be possible under the null hypothesis, and we would have a significant result.

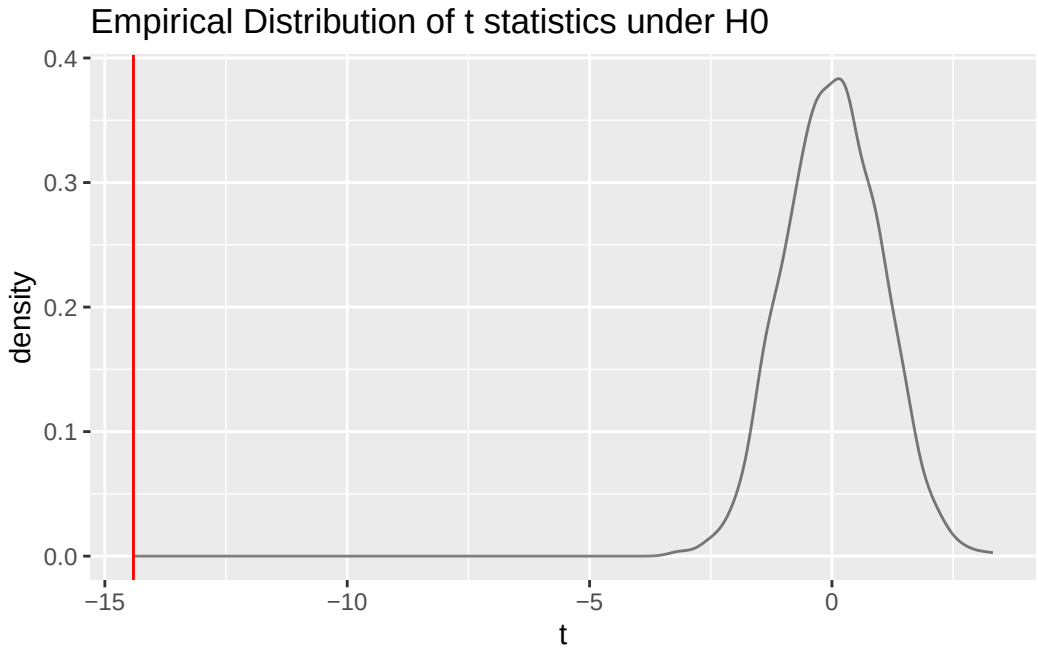
In this particular example, code to do this could look like this:

```
credit2 <- filter(credit, group != "NA")

t.test(age ~ group, data = credit2)
```

```
    Welch Two Sample t-test

data:  age by group
t = -14.41, df = 17863, p-value <2e-16
alternative hypothesis: true difference in means between group 0 and group 1 is not equal to
95 percent confidence interval:
 -2.93313 -2.23077
sample estimates:
mean in group 0 mean in group 1
        50.5875         53.1694
```

```
set.seed(500) #make the results reproducible
ttest <- do(1000) * (t.test(age ~ shuffle(group), data = credit2)$statistic)
ttest <- as.data.frame(ttest)
```

```
gf_dens(~ t, data = ttest) %>%
  gf_vline(xintercept = -14.41, color = "red") %>%
  gf_labs(title = "Empirical Distribution of t statistics under H0")
```




```
pdata(~ t, -14.41, data = ttest, lower.tail = TRUE)
```

```
[1] 0
```

In this particular case, it's clear that none of the statistics under the empirical null distribution are anywhere near our observed t from the original data. This suggests that the average age for the higher income group is above that of the average age of the lower income group.

Add 6

The diabetes data set (loaded below) is used to demonstrate ridge regression in Chapter 7. We want to try to verify the textbook results. Note that they did not use a training/test split so we are using the entire data set too.

```
diabetes <- read.csv("http://web.stanford.edu/~hastie/CASI_files/DATA/diabetes.csv",
                     header = TRUE)
diabetes <- select(diabetes, -X)
```

The data was pre-processed. The text states that the predictors were standardized to mean 0 and sum of squares 1, and the response had it's mean subtracted off (i.e. it was centered), but not scaled. So, we do that just to match (clumsily).

```
n <- nrow(diabetes)
diabetes2 <- mutate(diabetes, prog = scale(prog, scale = FALSE),
                    age = scale(age)/sqrt(n-1),
                    sex = scale(sex)/sqrt(n-1),
                    bmi = scale(bmi)/sqrt(n-1),
                    map = scale(map)/sqrt(n-1),
                    tc = scale(tc)/sqrt(n-1),
                    ldl = scale(ldl)/sqrt(n-1),
                    hdl = scale(hdl)/sqrt(n-1),
                    tch = scale(tch)/sqrt(n-1),
                    ltg = scale(ltg)/sqrt(n-1),
                    glu = scale(glu)/sqrt(n-1))
```

SOLUTION

(a) Use OLS to predict *prog* using *diabetes2*. Do you obtain the same results as in Table 7.3?

```
myControl <- trainControl(method = "repeatedcv",
                          number = 10,
                          repeats = 10)
```

```
# MLR Model
set.seed(240)
reglm_model1 <- train(
  prog ~ .,
  data = diabetes2,
  method = "lm",
  trControl = myControl
)

# How to get the final model
summary(reglm_model1)
```

```
Call:
lm(formula = .outcome ~ ., data = dat)
```

```
Residuals:
    Min      1Q  Median      3Q     Max
-155.83  -38.53   -0.23   37.81  151.35

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -5.40e-14   2.58e+00    0.00   1.0000
age         -1.00e+01   5.97e+01   -0.17   0.8670
sex         -2.40e+02   6.12e+01   -3.92   0.0001 ***
bmi          5.20e+02   6.65e+01    7.81  4.3e-14 ***
map          3.24e+02   6.54e+01    4.96  1.0e-06 ***
tc          -7.92e+02   4.17e+02   -1.90   0.0579 .
ldl          4.77e+02   3.39e+02    1.41   0.1604
hdl          1.01e+02   2.13e+02    0.48   0.6347
tch          1.77e+02   1.61e+02    1.10   0.2735
ltg          7.51e+02   1.72e+02    4.37  1.6e-05 ***
glu          6.76e+01   6.60e+01    1.02   0.3060
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 54.2 on 431 degrees of freedom
Multiple R-squared:  0.518, Adjusted R-squared:  0.507
F-statistic: 46.3 on 10 and 431 DF,  p-value: <2e-16
```

The model appears to match the coefficients in table 7.3, up to rounding. E.G. slope for sex is -239.8 in text and reported as -240 here. Later, when we print the coefficients, we will see we have the same ones.

(b) Explain what cross-validation is (generally), and how it can be used to help choose a lambda in ridge regression.

Cross-validation is where we take our data set (or training set) and split it into k roughly equal size pieces at random. We fit the model on k-1 pieces and test out it's performance on the last piece. We do this with every piece playing the "left-out" role once. At the end, we aggregate the assessment measures together in some way (sum or average). If you have multiple potential lambda values, you run CV with each lambda and at the end, you can compare the assessments in relation to each lambda. The best lambda has the lowest MSE (or could use MAE or R-squared).

(c) Use CV with ridge regression to select your lambda using an appropriate grid. What lambda is selected?

```r
# Ridge regression
myGrid <- expand.grid(alpha = 0, #fix this to make it ridge
                      lambda = 10^seq(10, -2, length = 100))

set.seed(240)
ridge_model1 <- train(
  prog ~ .,
  data = diabetes2,
  method = "glmnet",
  tuneGrid = myGrid,
  trControl = myControl
)
```

```
Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
: There were missing values in resampled performance measures.
```

```r
ridge_model1$bestTune$lambda
```

```
[1] 3.51119
```

The lambda chosen was 3.51119.

(d) Use ridge regression with lambda = 0.1. Do you obtain the same results as in Table 7.3?

You don't have to re-run this. You just ask for the coefficients at lambda = 0.1.

```r
coef(ridge_model1$finalModel, s = 0.1)
```

```
11 x 1 sparse Matrix of class "dgCMatrix"
                    s=0.1
(Intercept) -3.21421e-14
age         -1.96337e+00
sex         -2.18701e+02
bmi          5.04476e+02
map          3.09709e+02
tc          -1.19907e+02
ldl         -4.97036e+01
hdl         -1.80454e+02
tch          1.13547e+02
ltg          4.72886e+02
glu          8.06083e+01
```

The values here do not 100% match the ones in the textbook Table 7.3. The relative magnitudes are similar, e.g. bmi's slope here is 504 and its 489.7 in the text. The solution in the text appears to be at a slightly stronger lambda than 0.1, at least in terms of how this is coded.

(e) Implement LASSO with CV to select your lambda using an appropriate grid. What lambda is selected?

```
# Lasso regression
myGrid <- expand.grid(alpha = 1,
                      lambda = 10^seq(10, -2, length = 100))

set.seed(240)
lasso_model1 <- train(
  prog ~ .,
  data = diabetes2,
  method = "glmnet",
  tuneGrid = myGrid,
  trControl = myControl
)
```

```
Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
: There were missing values in resampled performance measures.
```

```
lasso_model1$bestTune$lambda
```

```
[1] 0.869749
```

The best lambda for Lasso was chosen as 0.8697.

(f) Implement a regression tree to predict *prog*. What cp corresponds to the default tree?

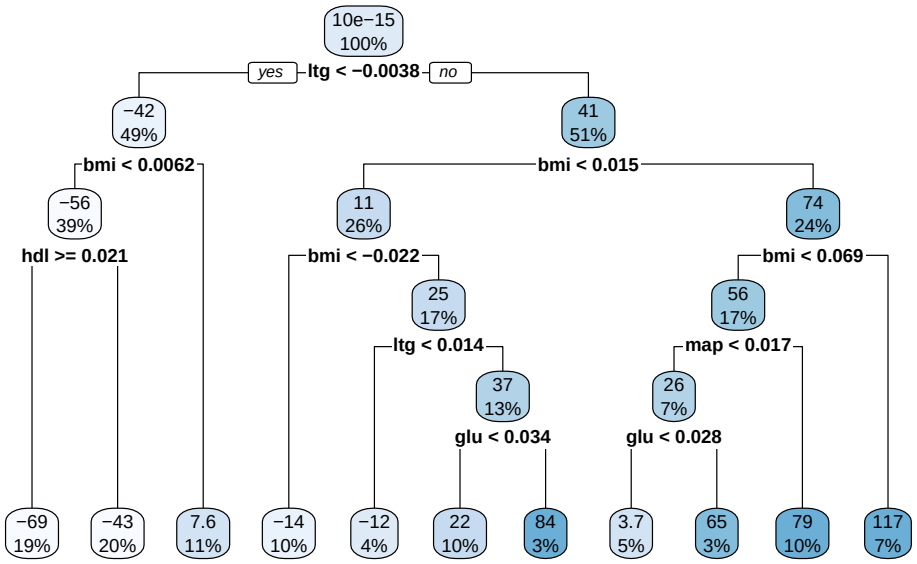
The default tree has a cp of 0.01.

```
set.seed(240) # for reproducibility
prog_tree <- train(
  prog ~ .,
  data = diabetes2,
  method = "rpart",
  trControl = myControl
)
```

```
Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
: There were missing values in resampled performance measures.
```

If we wanted to view the tree, we should fit it just with rpart outside of caret.

```
prog_tree2 <- rpart(prog ~ . , data = diabetes2)
rpart.plot(prog_tree2)
```




```
printcp(prog_tree2)
```

```
Regression tree:
rpart(formula = prog ~ ., data = diabetes2)

Variables actually used in tree construction:
[1] bmi glu hdl ltg map

Root node error: 2621009/442 = 5930

n= 442

        CP nsplit rel error xerror    xstd
```

```
1  0.29154      0    1.0000 1.0057 0.05048
2  0.08523      1    0.7085 0.8156 0.04928
3  0.05660      2    0.6232 0.7315 0.04814
4  0.03066      3    0.5666 0.6898 0.04469
5  0.02031      4    0.5360 0.6684 0.04430
6  0.01569      5    0.5157 0.6656 0.04392
7  0.01345      6    0.5000 0.6778 0.04470
8  0.01101      8    0.4731 0.6906 0.04613
9  0.01055      9    0.4621 0.7045 0.04733
10 0.01000     10    0.4515 0.7036 0.04691
```

(g) Explain why we don't need a *glm* here to predict *prog*.

prog is a continuous, numeric response. We use *glm*'s when the response is different from this - logistic for 0/1, poisson for counts, etc. So, we don't need a glm here - OLS is fine conceptually (though we didn't check conditions for it).

(h) Which model of these do you prefer? How do the fits differ? (Hint: Besides comparing coefficients/involved variables, you can compare MSEs.)

```
# Extract out of sample performance measures
summary(resamples(list(
  lm_model = reglm_model1,
  ridge_model = ridge_model1,
  lasso_model = lasso_model1,
  tree_model = prog_tree
)))
```

```
Call:
summary.resamples(object = resamples(list(lm_model = reglm_model1,
 ridge_model = ridge_model1, lasso_model = lasso_model1, tree_model
 = prog_tree)))

Models: lm_model, ridge_model, lasso_model, tree_model
Number of resamples: 100

MAE
               Min. 1st Qu.  Median    Mean 3rd Qu.    Max. NA's
lm_model    33.9006 41.0164 44.3645 44.3580 47.5678 55.2808    0
ridge_model 33.0216 41.2822 44.5242 44.3782 47.2710 55.8425    0
lasso_model 32.6397 41.3255 44.5954 44.4042 47.5351 55.6062    0
tree_model  41.9634 49.0227 52.1925 52.4720 55.2832 65.1033    0
```

```
RMSE
               Min. 1st Qu.  Median    Mean 3rd Qu.    Max. NA's
lm_model    42.4994 50.7816 54.4293 54.5087 57.8343 66.8996    0
ridge_model 41.5140 50.8948 54.6995 54.4991 57.6445 66.8927    0
lasso_model 40.8061 50.9242 54.5043 54.4674 57.7605 66.6004    0
tree_model  50.6084 60.3815 64.4331 64.7517 68.6650 81.3501    0

Rsquared
                Min.  1st Qu.   Median     Mean  3rd Qu.     Max. NA's
lm_model    0.2754664 0.446659 0.519065 0.507894 0.574720 0.696888    0
ridge_model 0.2749061 0.445267 0.515919 0.508282 0.574621 0.698555    0
lasso_model 0.2715524 0.440501 0.519795 0.508603 0.571710 0.691629    0
tree_model  0.0309684 0.232523 0.321605 0.313384 0.381514 0.551714    0
```

The tree is doing way worse than everything else. The other three are extremely similar in performance.

We look at the coefficients next. The tree doesn't have coefficients. You could do something where you just indicated if the tree used the variable or something instead if you wanted though.

```
# Grab the coefficients
reglm_coefs <- summary(reglm_model1)$coefficients[,1]
ridge_coefs <- coef(ridge_model1$finalModel, s = ridge_model1$bestTune$lambda)[,1]
lasso_coefs <- coef(lasso_model1$finalModel, s = lasso_model1$bestTune$lambda)[,1]

# Combine into a dataset
mydata <- data.frame(cbind(reglm_coefs, ridge_coefs, lasso_coefs))
# Keep the variable names for later stuff
mydata <- mydata |>
  # keep names in for later pivot
  mutate(name = row.names(mydata))

# Make a table
mydata |>
  select(reglm_coefs, ridge_coefs, lasso_coefs) |>
  kableExtra::kable(booktabs = TRUE)
```

	reglm_coefs	ridge_coefs	lasso_coefs
(Intercept)	0.0000	0.00000	0.00000

	reglm_coefs	ridge_coefs	lasso_coefs
age	-10.0122	-1.96338	0.00000
sex	-239.8191	-218.70102	-201.15874
bmi	519.8398	504.47617	522.60008
map	324.3904	309.70894	299.06829
tc	-792.1842	-119.90721	-112.23682
ldl	476.7458	-49.70356	0.00000
hdl	101.0446	-180.45431	-218.59393
tch	177.0642	113.54664	9.26496
ltg	751.2793	472.88600	516.12767
glu	67.6254	80.60825	55.95008

Here we computed the MSEs on the full data set since there is no training/test split. OLS actually has the best performance in this regard, so you could argue you want to stick with it, unless you want to remove some variables (which lasso does).

```
# Get predictions
regtest_pred <- predict(reglm_model1, diabetes2)
ridgetest_pred <- predict(ridge_model1, diabetes2, s = ridge_model1$bestTune$lambda)
lassotest_pred <- predict(lasso_model1, diabetes2, s = lasso_model1$bestTune$lambda)
treetest_pred <- predict(prog_tree, diabetes2)

# Compute MSEs
regtest_mse <- mean((diabetes2$prog - regtest_pred)^2)
regtest_mse
```

```
[1] 2859.69
```

```
ridgetest_mse <- mean((diabetes2$prog - ridgetest_pred)^2)
ridgetest_mse
```

```
[1] 2881.33
```

```
lassotest_mse <- mean((diabetes2$prog - lassotest_pred)^2)
lassotest_mse
```

```
[1] 2883.64
```

```
treetest_mse <- mean((diabetes2$prog - treetest_pred)^2)
treetest_mse
```

```
[1] 3695.69
```

Add 7

The Spam email data set (loaded below) is used to demonstrate logistic regression and classification trees in Chapter 8. Note that while you'd usually want a training/test set here, the book used the entire data set, so you should do that to try to verify their work.

```
spam <- read.csv("http://web.stanford.edu/~hastie/CASI_files/DATA/SPAM.csv",
                 header = TRUE)
```

SOLUTION

(a) Why should we not use OLS to predict *spam*?

The response is a 0/1 variable, so OLS doesn't make sense. It doesn't have a way to constrain the response values to be 0/1.

(b) Use logistic regression to predict *spam*. Verify you obtain the results in Table 8.3.

For this question, we don't use the caret *train* framework, but you should be able to understand code in that framework too (see above).

We'd usually want a training/test set idea here but to match the book, I'm using the entire data set.

The footnote reminds us that the X matrix was standardized. I'm assuming that means center = 0 and sd = 1. I'm hoping that thinking through these issues with scaling gives you a sense of why reproducibility is important. You should specify transformations you do CLEARLY so that others can replicate your results.

```
# remove id variable and scale all predictors
# this is done here by adding the response back in
# can also use a mutate targeted at only the numeric predictors
spamdata <- dplyr::select(spam, -spam, -testid)
spamdata <- data.frame(scale(spamdata))
spamdata <- mutate(spamdata, spam = factor(spam$spam))
```

With the variables appropriately transformed, we can fit the logistic regression.

```r
loglm <- glm(spam ~ . , data = spamdata, family = "binomial")
```

```
Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```r
summary(loglm)
```

```
Call:
glm(formula = spam ~ ., family = "binomial", data = spamdata)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -12.2653     1.9908   -6.16  7.2e-10 ***
make         -0.1189     0.0707   -1.68  0.09239 .
address      -0.1881     0.0894   -2.10  0.03536 *
all           0.0575     0.0556    1.03  0.30076
X3d           3.1412     2.1025    1.49  0.13517
our           0.3782     0.0685    5.52  3.3e-08 ***
over          0.2418     0.0684    3.53  0.00041 ***
remove        0.8919     0.1303    6.85  7.6e-12 ***
internet      0.2285     0.0675    3.39  0.00071 ***
order         0.2046     0.0794    2.58  0.00996 **
mail          0.0822     0.0468    1.76  0.07923 .
receive      -0.0515     0.0600   -0.86  0.39066
will         -0.1192     0.0638   -1.87  0.06177 .
people       -0.0240     0.0693   -0.35  0.72956
report        0.0485     0.0457    1.06  0.28885
addresses     0.3200     0.1878    1.70  0.08837 .
free          0.8577     0.1203    7.13  1.0e-12 ***
business      0.4262     0.1000    4.26  2.0e-05 ***
email         0.0639     0.0622    1.03  0.30453
you           0.1444     0.0622    2.32  0.02033 *
credit        0.5339     0.2744    1.95  0.05167 .
your          0.2905     0.0630    4.61  3.9e-06 ***
font          0.2065     0.1669    1.24  0.21584
X000          0.7865     0.1651    4.76  1.9e-06 ***
money         0.1887     0.0718    2.63  0.00853 **
hp           -3.2097     0.5228   -6.14  8.3e-10 ***
hpl          -0.9226     0.3899   -2.37  0.01797 *
george      -39.6236     7.1155   -5.57  2.6e-08 ***
X650          0.2399     0.1072    2.24  0.02526 *
```

```
lab          -1.4752     0.8909   -1.66  0.09774 .
labs         -0.1506     0.1432   -1.05  0.29297
telnet       -0.0687     0.1943   -0.35  0.72374
X857          0.8374     1.0787    0.78  0.43757
data         -0.4105     0.1733   -2.37  0.01784 *
X415          0.2200     0.5273    0.42  0.67649
X85          -1.0940     0.4196   -2.61  0.00912 **
technology    0.3719     0.1244    2.99  0.00280 **
X1999         0.0197     0.0743    0.27  0.79082
parts        -0.1317     0.0934   -1.41  0.15847
pm           -0.3760     0.1664   -2.26  0.02384 *
direct       -0.1066     0.1272   -0.84  0.40221
cs          -16.2716     9.6074   -1.69  0.09033 .
meeting      -2.0617     0.6429   -3.21  0.00134 **
original     -0.2791     0.1805   -1.55  0.12198
project      -0.9785     0.3292   -2.97  0.00295 **
re           -0.8016     0.1575   -5.09  3.6e-07 ***
edu          -1.3295     0.2447   -5.43  5.5e-08 ***
table        -0.1774     0.1266   -1.40  0.16096
conference   -1.1474     0.4603   -2.49  0.01267 *
ch.          -0.3143     0.1077   -2.92  0.00350 **
ch..1        -0.0509     0.0674   -0.75  0.45066
ch..2        -0.0719     0.0917   -0.78  0.43291
ch..3         0.2832     0.0728    3.89  0.00010 ***
ch..4         1.3120     0.1737    7.55  4.2e-14 ***
ch..5         1.0318     0.4780    2.16  0.03088 *
crl.ave       0.3803     0.5976    0.64  0.52451
crl.long      1.7771     0.4912    3.62  0.00030 ***
crl.tot       0.5116     0.1365    3.75  0.00018 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 6170.2  on 4600  degrees of freedom
Residual deviance: 1815.8  on 4543  degrees of freedom
AIC: 1932

Number of Fisher Scoring iterations: 13
```

Huzzah! It matches. The values in the text have been rounded to 2 decimals.

(c) Obtain a confusion matrix for the logistic regression. How well is the model doing?

About 39.4% of the data is spam. So we could get about 60.6% right just by saying it's all not spam. We want to do better than that.

```
#true class goes second
glmpred <- round(predict(loglm, spamdata, type = "response"))
spamtruth <- ifelse(spamdata$spam == TRUE, 1, 0)
# really useful function so you don't make the table manually
# predictions go first, then truth
confusionMatrix(factor(glmpred), factor(spamtruth))
```

```
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0 2666  194
         1  122 1619

               Accuracy : 0.931
                 95% CI : (0.924, 0.938)
    No Information Rate : 0.606
    P-Value [Acc > NIR] : < 2e-16

                  Kappa : 0.855

 Mcnemar's Test P-Value : 6.5e-05

            Sensitivity : 0.956
            Specificity : 0.893
         Pos Pred Value : 0.932
         Neg Pred Value : 0.930
             Prevalence : 0.606
         Detection Rate : 0.579
   Detection Prevalence : 0.622
      Balanced Accuracy : 0.925

       'Positive' Class : 0
```

Overall, we are getting 93.1% correct. So, the model is doing very well.

(d) Use a classification tree to predict *spam*. Attempt to obtain the tree in Figure 8.7. Note that you may need to prune or adjust control parameters to obtain this tree. If you cannot obtain the tree, obtain one you want to work with for part e below.

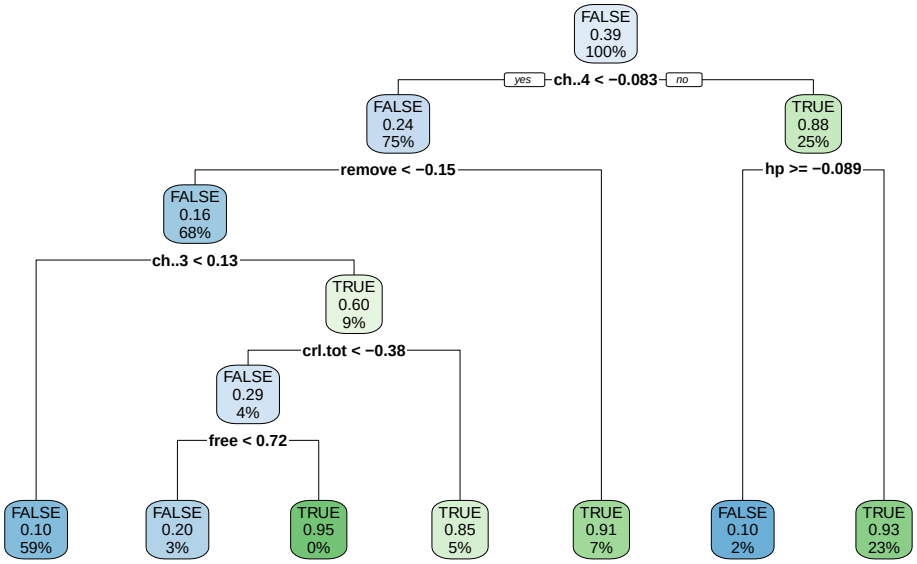
The text says it used rpart, but doesn't specify how the variables were standardized. We leave them as above, continuing with the spamdata data set.

From here, it looks like they may have indeed just used the default tree grown by rpart. We can check if we view the tree.

```
set.seed(495)
spam.rpart <- rpart(factor(spam) ~ ., method ="class", data = spamdata)
printcp(spam.rpart)
```

```
Classification tree:
rpart(formula = factor(spam) ~ ., data = spamdata, method = "class")

Variables actually used in tree construction:
[1] ch..3   ch..4   crl.tot free    hp      remove

Root node error: 1813/4601 = 0.394

n= 4601

       CP nsplit rel error xerror    xstd
1 0.47656      0    1.0000 1.0000 0.01828
2 0.14892      1    0.5234 0.5433 0.01535
3 0.04302      2    0.3745 0.4468 0.01425
4 0.03089      4    0.2885 0.3271 0.01254
5 0.01048      5    0.2576 0.2796 0.01172
6 0.01000      6    0.2471 0.2653 0.01145
```

```
rpart.plot(spam.rpart)
```



Indeed! Default tree. Note that the CV error and such is off, but extremely consistent, as we don't know what seed they used when that was being evaluated.

(e) Obtain a confusion matrix for your classification tree. How do your error rates compare to those reported in Figure 8.7?

```
treepred <- predict(spam.rpart, spamdata, type = "class")
confusionMatrix(treepred, spamdata$spam)
```

```
Confusion Matrix and Statistics

          Reference
Prediction FALSE TRUE
     FALSE  2654  314
     TRUE    134 1499

               Accuracy : 0.903
                 95% CI : (0.894, 0.911)
    No Information Rate : 0.606
    P-Value [Acc > NIR] : <2e-16

                  Kappa : 0.793
```

```
 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.952
            Specificity : 0.827
         Pos Pred Value : 0.894
         Neg Pred Value : 0.918
             Prevalence : 0.606
         Detection Rate : 0.577
   Detection Prevalence : 0.645
      Balanced Accuracy : 0.889

       'Positive' Class : FALSE
```

```
134/(2654+134)
```

```
[1] 0.0480631
```

```
314/(1499+314)
```

```
[1] 0.173194
```

Overall, the tree is about 90.3 percent accurate. We have training error of 4.8 percent on not spam and 17ish percent on spam.

This is overly optimistic. We can look at the CV error in the output to see a better estimate of error.

```
0.394045 * 0.274683
```

```
[1] 0.108237
```

The estimated error is 10.83 percent, which is still not bad. (So accuracy of 89.17 percent.) We don't have a similar value to compare to for the logistic regression, since we didn't split into training/test. It's not clear if they did training/test here using the testid variable, but they had different error rates in Figure 8.7.

(f) Which method do you prefer for predicting *spam*?

The tree has slightly worse performance but is MUCH easier to understand. It requires fewer variables. So I'd probably go with the tree, though I might grow it out a bit more if I find that's helpful.

CASI 8.6

```
galaxy <- read.table("http://web.stanford.edu/~hastie/CASI_files/DATA/galaxy.txt",
                     header = TRUE)
```

Data set description: Table of counts of galaxies binned into categories defined by redshift and magnitude. The column labels are log-redshift values, and the row labels magnitude.

Fit the Poisson regression model (8.39) to the galaxy data.

(Note that some data wrangling is required based on how the data is provided.)

SOLUTION

```
# add m values to the data set
galaxy2 <- mutate(galaxy, m = 18:1)

# pivot to get the log_r values in a variable
galaxydata <- galaxy2 %>%
  pivot_longer(-m, names_to = "log_r", values_to = "count")

# add r values 1:15 repeated
galaxydata <- mutate(galaxydata, r = c(rep(1:15, 18)))
```

This is slightly odd, as they are using the bin numbers, not actual variable values in the models, at least, based on their description. You CAN set up (I started to) the wrangling to get the actual m and r values back. Either way, the idea here is to fit a Poisson regression to verify you can read the output, etc.

Now we can go fit the model.

```
myglm <- glm(count ~ r*m + I(r^2) + I(m^2), data = galaxydata, family = poisson)
summary(myglm)
```

```
Call:
glm(formula = count ~ r * m + I(r^2) + I(m^2), family = poisson,
    data = galaxydata)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -1.73283    0.51229   -3.38  0.00072 ***
```

```
r           -0.21002    0.07184   -2.92  0.00346 **
m            0.34221    0.08164    4.19  2.8e-05 ***
I(r^2)      -0.02367    0.00336   -7.04  1.9e-12 ***
I(m^2)      -0.01444    0.00360   -4.02  5.9e-05 ***
r:m          0.03916    0.00484    8.10  5.6e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 903.68  on 269  degrees of freedom
Residual deviance: 230.32  on 264  degrees of freedom
AIC: 647.3

Number of Fisher Scoring iterations: 6
```

This is a complete second-order model.

```
lmtest::lrtest(myglm)
```

```
Likelihood ratio test

Model 1: count ~ r * m + I(r^2) + I(m^2)
Model 2: count ~ 1
  #Df LogLik Df Chisq Pr(>Chisq)
1   6 -317.7
2   1 -654.4 -5 673.4     <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

It looks like the overall model is significant.

Add 8

```
data(Credit)
Credit <- dplyr::select(Credit, -ID)
```

You may want to look over the help file for this data set. Note that the number of observations stated there is inaccurate. Our goal is to predict Balance. Do not worry about transforming Balance or any of the other predictors here.

SOLUTION

(a) Create an appropriate training/test split using a 75/25 percent split.

```
set.seed(123)
split_Cred  <- initial_split(Credit, prop = 0.75,
                             strata = "Balance")
train_Cred  <- training(split_Cred)
test_Cred   <- testing(split_Cred)
```

(b) Perform a regression using an automated variable selection algorithm of your choice and choose a model based on an appropriate descriptive statistic. Compute the test MSE.

```
myControl <- trainControl(method = "repeatedcv", number = 10, repeats = 10)
```

```
set.seed(240)
steplm_model1 <- train(
  Balance ~ .,
  data = train_Cred,
  method = "lmStepAIC",
  trace = FALSE, # suppresses output
  trControl = myControl
)

steplm_model1
```

```
Linear Regression with Stepwise Selection

299 samples
 10 predictor

No pre-processing
Resampling: Cross-Validated (10 fold, repeated 10 times)
Summary of sample sizes: 267, 271, 268, 269, 270, 269, ...
Resampling results:

  RMSE     Rsquared  MAE
  101.826  0.952447  80.9901
```

```
# How to get the final model
summary(steplm_model1)
```

```
Call:
lm(formula = .outcome ~ Income + Limit + Rating + Cards + StudentYes,
    data = dat)

Residuals:
   Min     1Q Median     3Q    Max
-185.6  -76.0  -13.4   53.2  326.0

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -512.0670    22.7513  -22.51  < 2e-16 ***
Income        -7.8652     0.2717  -28.95  < 2e-16 ***
Limit          0.2143     0.0373    5.74  2.3e-08 ***
Rating         0.7860     0.5569    1.41  0.15921
Cards         16.8362     4.9031    3.43  0.00068 ***
StudentYes   432.3033    20.5840   21.00  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 100 on 293 degrees of freedom
Multiple R-squared:  0.953, Adjusted R-squared:  0.952
F-statistic: 1.18e+03 on 5 and 293 DF,  p-value: <2e-16
```

```r
# Get predictions on test set
steptest_pred <- predict(steplm_model1, test_Cred)
steptest_mse <- mean((test_Cred$Balance - steptest_pred)^2)
steptest_mse
```

```
[1] 9342.52
```

(c) Perform the lasso and choose a model based on an appropriate descriptive statistic. Compute the test MSE.

```r
# alpha = 1 is LASSO
myGrid <- expand.grid(alpha = 1,
                      lambda = 10^seq(10, -2, length = 100))

set.seed(240)
lasso_model1 <- train(
  Balance ~ .,
  data = train_Cred,
```

```
  method = "glmnet",
  tuneGrid = myGrid,
  trControl = myControl
)
```

```
Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
: There were missing values in resampled performance measures.
```

Look at the results.

```
#lasso_model1

# Get the complete solution path
# one side is lambda = infinity, the other is lambda = 0
# be sure you understand which is which and why
#plot(lasso_model1$finalModel)

# go get the best lambda
lasso_model1$bestTune$lambda
```

```
[1] 0.497702
```

```
# to get the coefficients for the final model
# you have to tell it the lambda you want the coefs at!
# and s is the lambda value
# The . means the coefficient has been set to 0
coef(lasso_model1$finalModel, s = lasso_model1$bestTune$lambda)
```

```
12 x 1 sparse Matrix of class "dgCMatrix"
                     s=0.497702
(Intercept)        -486.431021
Income               -7.772393
Limit                 0.195937
Rating                1.040764
Cards                15.057902
Age                  -0.312329
Education            -1.006097
GenderFemale         -2.069250
StudentYes          427.573713
MarriedYes          -13.715191
EthnicityAsian       17.162174
EthnicityCaucasian   16.724511
```

```
# Get predictions on test set
lassotest_pred <- predict(lasso_model1, test_Cred, s = lasso_model1$bestTune$lambda)
lassotest_mse <- mean((test_Cred$Balance - lassotest_pred)^2)
lassotest_mse
```

```
[1] 9261.46
```

(d) Fit an elastic-net penalty with alpha = 0.5, and choose a model based on an appropriate descriptive statistic. Compute the test MSE.

```
# alpha is specified
myGrid <- expand.grid(alpha = 0.5,
                      lambda = 10^seq(10, -2, length = 100))

# This will take some time to run because of all the alpha values AND lambda values
set.seed(240)
enet_model1 <- train(
  Balance ~ .,
  data = train_Cred,
  method = "glmnet",
  tuneGrid = myGrid,
  trControl = myControl
)
```

```
Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
: There were missing values in resampled performance measures.
```

Look at the results.

```
#enet_model1

# Get the complete solution path
# one side is lambda = infinity, the other is lambda = 0
# be sure you understand which is which and why
#plot(enet_model1$finalModel)

# go get the best alpha and lambda
enet_model1$bestTune$alpha
```

```
[1] 0.5
```

```r
enet_model1$bestTune$lambda
```

```
[1] 0.657933
```

```r
# to get the coefficients for the final model
# you have to tell it the lambda you want the coefs at!
# and s is the lambda value
# The . means the coefficient has been set to 0
coef(enet_model1$finalModel, s = enet_model1$bestTune$lambda)
```

```
12 x 1 sparse Matrix of class "dgCMatrix"
                     s=0.657933
(Intercept)        -492.548216
Income               -7.790928
Limit                 0.178480
Rating                1.304563
Cards                13.999246
Age                  -0.329812
Education            -1.018788
GenderFemale         -2.900608
StudentYes          427.581656
MarriedYes          -15.164230
EthnicityAsian       19.860657
EthnicityCaucasian   18.583214
```

```r
# Get predictions on test set
enettest_pred <- predict(enet_model1, test_Cred, s = enet_model1$bestTune$lambda)
enettest_mse <- mean((test_Cred$Balance - enettest_pred)^2)
enettest_mse
```

```
[1] 9219.83
```

(e) How do your models compare? Which model do you prefer? Why? (Should be able to compare coefficients.)

```r
# Grab the coefficients
enet_coefs <- coef(enet_model1$finalModel, s = enet_model1$bestTune$lambda)[,1]
lasso_coefs <- coef(lasso_model1$finalModel, s = lasso_model1$bestTune$lambda)[,1]

# This is missing coefs for ones set to 0.
```

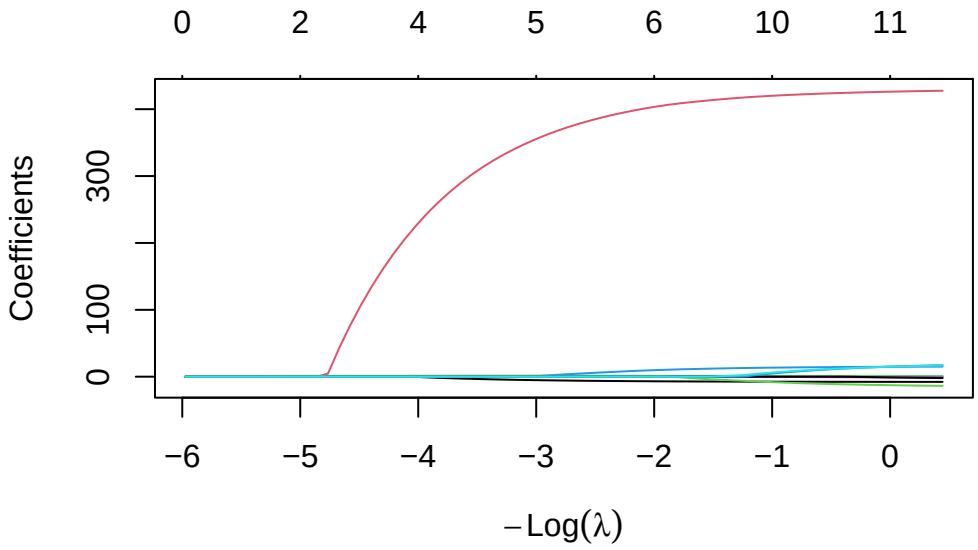
```r
steplm_coefs <- summary(steplm_model1)$coefficients[,1]

#Slightly clunky but will fix it
var_names <- names(enet_coefs)
steplm_alignedcoefs <- setNames(numeric(length(var_names)), var_names)
common_terms <- intersect(var_names, names(steplm_coefs))
steplm_alignedcoefs[common_terms] <- steplm_coefs[common_terms]

# Combine into a dataset
mydata <- data.frame(cbind(steplm_alignedcoefs, lasso_coefs, enet_coefs))
# Keep the variable names for later stuff
mydata <- mydata |>
  # keep names in for later pivot
  mutate(name = row.names(mydata)) |>
  rename(steplm_coefs = steplm_alignedcoefs)

# Make a table
mydata |>
  dplyr::select(steplm_coefs, lasso_coefs, enet_coefs) |>
  kable(booktabs = TRUE)
```

	steplm_coefs	lasso_coefs	enet_coefs
(Intercept)	-512.067013	-486.431021	-492.548216
Income	-7.865185	-7.772393	-7.790928
Limit	0.214258	0.195937	0.178480
Rating	0.786025	1.040764	1.304563
Cards	16.836152	15.057902	13.999246
Age	0.000000	-0.312329	-0.329812
Education	0.000000	-1.006097	-1.018788
GenderFemale	0.000000	-2.069250	-2.900608
StudentYes	432.303251	427.573713	427.581656
MarriedYes	0.000000	-13.715191	-15.164230
EthnicityAsian	0.000000	17.162174	19.860657
EthnicityCaucasian	0.000000	16.724511	18.583214

The elastic net has the lowest test MSE. Neither the lasso or elastic net set any predictor coefficients to 0. So, if we wanted a model with fewer variables, we'd use the stepwise model. Otherwise, the elastic net has the best predictive performance.

(f) Plot the complete lasso solution path. Describe what happens in the figure.

```
plot(lasso_model1$finalModel)
```



At the left, lambda is large. At right, we have the OLS solution, lambda is 0. Each variable is a line. As we move from left to right, the lambda decreases and variables enter the model. The numbers at the top are the current number of variables in the model at that lambda value. These would be the "active set" of variables in the model at the time. Between each active set change, the B's are linear, making the entire path easy to solve for.

Add 9

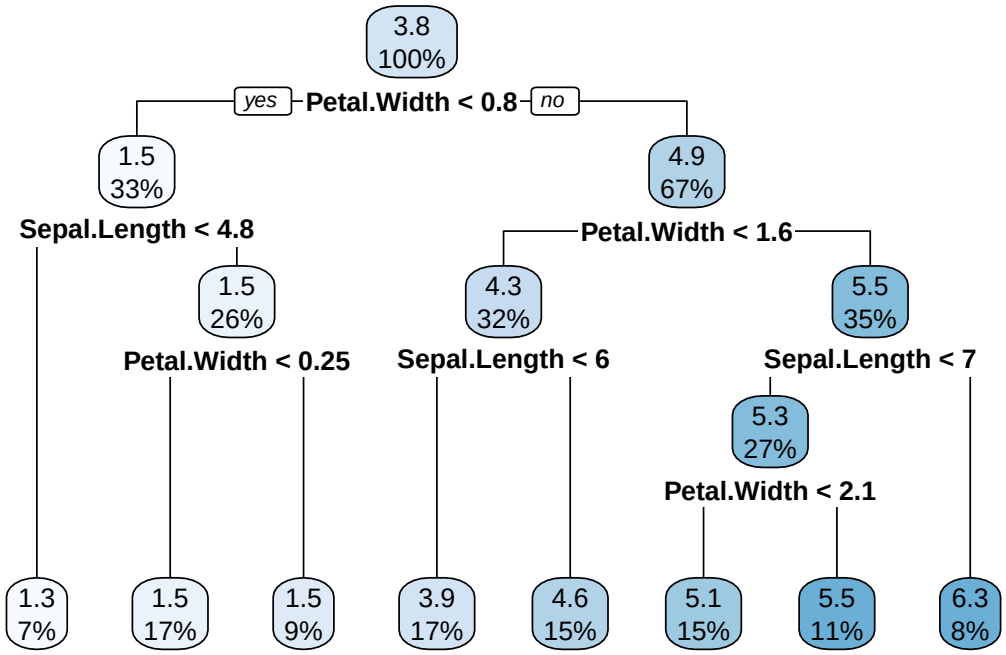
The tree here is fit with rpart outside of the *train* framework from *caret* so we can play with other tuning parameters.

```
data(iris)
glimpse(iris)
```

```
Rows: 150
Columns: 5
$ Sepal.Length <dbl> 5.1, 4.9, 4.7, 4.6, 5.0, 5.4, 4.6, 5.0, 4.4, 4.9, 5.4, 4.~
$ Sepal.Width  <dbl> 3.5, 3.0, 3.2, 3.1, 3.6, 3.9, 3.4, 3.4, 2.9, 3.1, 3.7, 3.~
$ Petal.Length <dbl> 1.4, 1.4, 1.3, 1.5, 1.4, 1.7, 1.4, 1.5, 1.4, 1.5, 1.5, 1.~
```

```
$ Petal.Width  <dbl> 0.2, 0.2, 0.2, 0.2, 0.2, 0.4, 0.3, 0.2, 0.2, 0.1, 0.2, 0.~
$ Species      <fct> setosa, setosa, setosa, setosa, setosa, setosa, setosa, s~
```

```
# for parts a and b
iris.control <- rpart.control(minbucket = 10, minsplit = 30, cp = 0)
iris.rpart <- rpart(Petal.Length ~ . - Species, data = iris, method = "anova",
                    control = iris.control)
rpart.plot(iris.rpart)
```




```
# for part c
iris[121,]
```

```
    Sepal.Length Sepal.Width Petal.Length Petal.Width   Species
121          6.9         3.2          5.7         2.3 virginica
```

```
# to help with part d sketch
favstats(~ Petal.Width, data = iris)
```

```
 min  Q1 median  Q3 max    mean       sd   n missing
 0.1 0.3    1.3 1.8 2.5 1.19933 0.762238 150       0
```

```
favstats(~ Sepal.Length, data = iris)
```

```
 min  Q1 median  Q3 max    mean       sd   n missing
 4.3 5.1    5.8 6.4 7.9 5.84333 0.828066 150       0
```

SOLUTION

(a) Is this a classification or a regression tree? How do you know?

This is a regression tree. We know the response is numeric and we see method = "anova" in the rpart command.

(b) Explain what the minbucket and minsplit control options do.

Minbucket and minsplit are both control options to govern how the tree is created. Minbucket says that when a split occurs, the resulting nodes must have at least 10 observations in them. Minsplit says that in order for a split to occur, the node being split must have at least 30 observations in it. Changing these values will cause different trees to be built.

(c) What is the residual for observation 121? (Remember residual = observed - predicted response value)

In order to get the residual, we have to track the observation through the tree. Observation 121 has a Petal.Width of 2.3, so it goes right at the first split, and then right at the second split. At the next split, we need it's Sepal.Length, which is 6.9. That is less than 7, so it goes left at that split, and then goes right because it's Petal.Width of 2.3 is > 2.1. Thus, it ends up in the node with a response prediction of 5.5 (11 percent of observations are in that node).

To get the residual, we note it's actual Petal.Length which is 5.7. So our residual is 0.2.

```
5.7-5.5
```

```
[1] 0.2
```

(d) Only 2 variables are used in the tree. Sketch the hypercubes formed in 2-D space, labeling them with their predicted response values.

See sketch. (Will append.)

Add 9 Practice Mid term Solutions

Hyper cube Visual (Rough Sketch)